```
class Customer < ActiveRecord::Base
  has_many :line_items, -> { group 'orders.id' },
                        through: :orders
end
```

#### 4.3.3.4 includes

You can use the `includes` method to specify second-order associations that should be eager-loaded when this association is used. For example, consider these models:

```
class Customer < ActiveRecord::Base
  has_many :orders
end

class Order < ActiveRecord::Base
  belongs_to :customer
  has_many :line_items
end

class LineItem < ActiveRecord::Base
  belongs_to :order
end
```

If you frequently retrieve line items directly from customers (`@customer.orders.line_items`), then you can make your code somewhat more efficient by including line items in the association from customers to orders:

```
class Customer < ActiveRecord::Base
  has_many :orders, -> { includes :line_items }
end

class Order < ActiveRecord::Base
  belongs_to :customer
  has_many :line_items
end

class LineItem < ActiveRecord::Base
  belongs_to :order
end
```

#### 4.3.3.5 limit

The `limit` method lets you restrict the total number of objects that will be fetched through an association.

```
class Customer < ActiveRecord::Base
  has_many :recent_orders,
    -> { order('order_date desc').limit(100) },
    class_name: "Order",
end
```

#### 4.3.3.6 offset

The `offset` method lets you specify the starting offset for fetching objects via an association. For example, `-> { offset(11) }` will skip the first 11 records.

#### 4.3.3.7 order

The `order` method dictates the order in which associated objects will be received (in the syntax used by an SQL `ORDER BY` clause).

```
class Customer < ActiveRecord::Base
  has_many :orders, -> { order "date_confirmed DESC" }
end
```

#### 4.3.3.8 readonly

If you use the `readonly` method, then the associated objects will be read-only when retrieved via the association.

#### 4.3.3.9 select

The `select` method lets you override the SQL `SELECT` clause that is used to retrieve data about the associated objects. By default, Rails retrieves all columns.